

I Built My Entire UX Portfolio Website Using AI — Here's Exactly How It Happened

Danny Driscoll, UX Designer | displayedux.com

March 2026

The Moment I Realized This Was Actually Going to Work

About three hours into the first build session, Claude had scaffolded a Next.js project, created a shared `CaseStudyTemplate` component, and was generating the TeleSign case study page from a content brief I'd written. The page had a hero image, a bento stat grid with outcome metrics, a YouTube video embed, and a mobile-responsive layout — and I hadn't written a single line of code.

I've been a UX designer for 18 years. I know what good design looks like. I know what an information architecture should feel like. I can write a brief, define a component, and critique a layout at a glance. What I've never been able to do is *build* fast enough to keep up with my own thinking. That gap — between what I can imagine and what I can ship — is where portfolios go to die.

Claude in Cowork mode just closed that gap. Here's the full story of how it happened.

Background: 18 Years of Work, No Good Place to Show It

I'm a product designer. I've worked on mobile apps for Netflix, communications platforms for TeleSign, college search experiences for what is now Appily.com, and enterprise self-service portals for B2B SaaS companies. The work is strong. The results are real. But my portfolio had been a half-finished WordPress site for years — always *almost* ready, never quite shipped.

The problem was never content. I had six detailed case studies written and documented, complete with outcome metrics, screenshots, and video walkthroughs. The problem was execution. Every time I sat down to build a proper portfolio site, I'd get three hours in, hit a CSS issue, lose momentum, and tell myself I'd come back to it.

I needed to build something that looked professional, loaded fast, and actually showcased the depth of each case study — not just a thumbnail and a paragraph. I needed a site that worked for both hiring managers and potential clients. And I needed to ship it.

The Tools: Claude, Cowork Mode, Next.js, and MCPs

Let me break down what I actually used, in plain language.

Claude is Anthropic's AI. Most people know it as a chat assistant. What I used it for is closer to a software development partner — one that can read files, write code, take screenshots, run commands, and make decisions across a long-running project.

Cowork mode is a desktop app built on top of Claude. It's aimed at non-developers who want to automate file and task management. Think of it as Claude with hands — it can actually touch your computer, not just give you instructions to follow yourself.

Next.js 14 is the framework we built the site on. App Router architecture, TypeScript for type safety, Tailwind CSS for styling. This is a production-grade stack used by major tech companies. I didn't need to know how it worked in detail. I needed to know *what* I wanted built.

MCPs — Model Context Protocol plugins — are the real game-changer. These are installable extensions that give Claude new capabilities. The ones we used:

- **Filesystem MCP** — Direct read/write access to my project files
- **Computer Use MCP** — Screenshots and desktop control for visual QA
- **Claude in Chrome MCP** — DOM-aware browser automation (clicking, inspecting, form-filling)
- **Playwright MCP** — Headless browser testing
- **Dispatch MCP** — Spawning multiple parallel Claude agents for concurrent work
- **Context Mode MCP** — Indexing and searching large codebases semantically

Put all of this together and you have a system where Claude can write a component, deploy it to Netlify, take a screenshot of the live page, identify a mobile layout issue, write a fix, and re-deploy — without me touching anything but the review.

The Build: Phase by Phase

Discovery and Planning

Before writing any code, Claude and I worked through the site architecture together. I described what I needed: six case study pages with a consistent structure, a homepage that established credibility quickly, an about page, and a contact mechanism. I gave Claude links to a few portfolio sites I admired and asked for an honest assessment.

The analysis was sharp. Claude identified what made those sites work (strong information hierarchy, specificity in outcome metrics, judicious use of video), and what was missing (too many sites used identical layouts that made every designer look the same). That became the brief: differentiate with depth, not decoration.

Technical Setup

Claude initialized a Next.js 14 project with App Router, configured TypeScript and Tailwind, set up the GitHub repository on a `design-v2` branch, and connected Netlify for auto-deployment. Total time from blank folder to deployed placeholder: about 40 minutes.

There was one workflow I didn't anticipate that turned out to be critical: the **git worktree pattern**. Because the Filesystem MCP only has write access to git worktrees (not the main repository checkout), all code edits happen inside a worktree. Claude would create a feature branch worktree, make changes, commit them, and then merge back to `design-v2` before pushing. Every change was tracked in git. Every deploy was reproducible.

This turned out to be a feature, not a limitation. It meant I had a clean commit history, no accidental overwrites, and a rollback path for every change.

Component Architecture

The core insight was that six case studies with identical structure shouldn't mean six separately-coded pages. Claude built a `CaseStudyTemplate` component that accepts a configuration object — sections, content blocks, YouTube video IDs, stat grids — and renders the full page. Each case study page is just a thin config wrapper around that shared component.

This took maybe a day to get right (there were TypeScript prop errors that needed sorting, and getting the bento stat grid to render correctly at all viewport sizes took a few iterations). But once it was right, adding a new case study page was a 20-minute job.

Content Integration

This is where the bulk of the work happened. For each case study, I provided Claude with a content brief: the project context, key metrics, screenshots, video links, and narrative arc. Claude drafted the copy, structured the sections, embedded the videos, and built the bento grids.

The YouTube integration deserves a callout. I had eight videos across six case studies — walkthroughs, demo recordings, design reviews. Claude extracted the video IDs from the URLs, built a `YouTubeEmbed` component with a responsive 16:9 container, and embedded them with proper titles and captions. Not one of them broke. Every single embed loads cleanly at full width on desktop and collapses gracefully on mobile.

One project mid-build: the college search platform I worked on had recently rebranded from Cappex to Appily.com. This meant every reference to "Cappex" across six files, multiple components, and the site metadata needed updating. Claude ran a search across the entire codebase via Context Mode MCP, replaced every instance of "Cappex" with "Appily.com," and added "formerly Cappex" notation near legacy screenshots (so the historical context wasn't lost). Start to finish: about eight minutes. Manually,

this would have been an afternoon of careful find-and-replace across multiple files, with a high chance of missing something.

What Broke (and How We Fixed It)

I want to be honest about this section because most AI build posts skip the failures.

The mobile hero images were a mess. Out of the box, several case study hero images were cropping in weird ways on iPhone-sized viewports — faces cut off, key UI elements disappearing. The standard Tailwind responsive utilities weren't helping because the `CaseStudyTemplate` component was using inline styles for image rendering, and inline styles override Tailwind classes. Claude's fix was to add a `<style>` block with `@media` queries that targeted specific CSS classes. This is a pattern worth remembering: when components use inline styles, media queries in a style block beat Tailwind every time.

The bento grid overflowed on narrow screens. Stat cards in the bento grid were breaking out of their containers on small viewports, causing horizontal scroll. Fix: `overflow-x: hidden` on the container, `flex-wrap: wrap` inside the grid, and minimum width constraints on the cards. Two lines of CSS, five minutes.

Hyphenation in stat cards. Long words in the outcome metric cards were breaking layout on narrow viewports. Added `hyphens: auto` and `word-break: break-word`. Simple fix, but annoying to spot if you're not doing real viewport QA.

Context window exhaustion. This one is more philosophical. Claude's context window — the amount of information it can actively hold during a session — has limits. On a long day of building, the session starts to get "compacted," which means older instructions get summarized and some nuance is lost. The fix was creating a `HANDOFF.md` file in the repository root at the end of every significant session. This document captures exactly what was done, what files were changed, what issues are open, and what to do next. Starting a new session by reading the handoff doc gives Claude the context it needs without relying on memory that doesn't persist.

The MCPs That Changed Everything

I've already described the MCPs above, but let me be specific about which ones had the highest leverage on this project.

Claude in Chrome MCP was transformative for visual QA. Instead of me manually resizing my browser and taking screenshots, Claude would navigate to each page, resize to iPhone and iPad dimensions, take screenshots, and return a visual assessment. When something looked wrong, it

described what was wrong and wrote the fix. QA loops that would have taken me an hour took ten minutes.

Dispatch MCP was transformative for parallel work. When it came time to integrate content across all six case studies, I could dispatch three parallel Claude agents — each handling two case studies simultaneously. The wall-clock time for content integration dropped by roughly 60%.

Context Mode MCP made the Appily.com rename trivial and would have been essential on a larger codebase. Being able to semantically search "all usages of Cappex" across 40+ files, rather than doing grepping by hand, is a genuine productivity shift.

Results

The site is live at displayedux.com.

Six case studies, each with meaningful depth: problem framing, design process, key decisions, outcome metrics, and embedded video. Mobile-responsive across all screen sizes. Fast — Next.js static generation means sub-second load times. Clean design that puts the work first and doesn't distract.

The case studies showcase work I'm genuinely proud of across 18 years: large-scale B2B SaaS redesigns, consumer mobile experiences, developer-facing API products, and high-stakes enterprise UX. Having all of it in one well-organized place, with consistent presentation, is something I've been trying to achieve for a long time.

Total build time across active working sessions: roughly 3 weeks at a pace of a few hours per day. This included discovery, architecture, component building, all six case study integrations, mobile QA, and deployment. A traditional dev-assisted build of the same scope would have taken 8-12 weeks and a significant budget.

What This Means for Designers

There's a version of this story that's threatening: *AI can build websites, so what do designers do?* I want to push back on that framing directly.

What made this project work was not Claude's ability to write JavaScript. It was my ability to write a clear brief, evaluate what was generated, identify the gap between what was built and what was right, and redirect with precision. Every meaningful design decision — the information architecture, the component structure, the content hierarchy, the visual tone, the QA criteria — came from 18 years of knowing what good looks like.

Claude is extremely good at execution. It is not good at caring whether the design is *right*. It doesn't have the taste, the professional accountability, or the user empathy to make the judgment calls that separate a good portfolio from a mediocre one. That part is still entirely human.

What changes is the ratio of time spent thinking versus time spent doing. For most of my career, building took 10x longer than designing. With this workflow, that ratio is closer to 1:1. That means more time designing, more iterations, more polish, and faster time to market for clients.

If you're a designer who's been avoiding "the technical stuff" — this is the moment to reconsider. You don't need to learn to code. You need to learn to direct.

What's Next

displayedux.com is version one. On my radar:

- A blog section (this post might be the first entry)
- Case study filtering by industry and deliverable type
- An engagement model page for clients interested in this AI-assisted build process
- A second client site built using the same playbook — to prove this isn't a one-time fluke

I've documented the full process as a reusable guide: the component patterns, the MCP workflow, the git worktree pattern, the template prompt for starting new client projects. The plan is to use this exact approach to build websites for other UX designers and small product teams who need something professional without a six-month engineering engagement.

If that sounds like something you need, reach out: d2drisco@icloud.com.

Full blog post will be hosted at [TBD] — follow me on LinkedIn for the link when it goes live.

Danny Driscoll is a UX designer with 18+ years of product design experience across enterprise SaaS, consumer mobile, and developer tools. He's based in the US and available for select freelance and consulting engagements. displayedux.com